Universität Rostock

gegründet 1419

OBJEKTE

Joern Ploennigs · AI4SC

cf. 'Bauhaus line drawing' by Walter Gropius

Zielsetzung

- Objekte als **Entwurfseinheiten** verstehen
- Domänen in Entitäten, Attribute und Verantwortlichkeiten zerlegen
- Beziehungen zwischen Objekten explizit im Entwurf festhalten
- UML-Klassendiagramm als Entwurfsartefakt und KI-Prompt nutzen
- Den Unterschied zwischen Datentypen und Klassen sowie zwischen Funktionen und Methoden verstehen

OBJEKTE UND KLASSEN

Objekte als Entwurfseinheiten

- Objekte sind Entwurfseinheiten **vor** dem Code
- Eine Domäne wird in Entitäten zerlegt, jede mit Attributen und Verantwortlichkeiten
- Eine Klasse beschreibt Struktur und Verantwortlichkeit einer Entität
- Beziehungen zwischen Klassen müssen explizit entworfen werden

Warum Objekte — Das syntaktische Problem

Objekte legen fest, wie sich wiederholende Datenstrukturen gespeichert werden. Dabei geht es darum, dass die Syntax der Datenstruktur eindeutig ist.

- Wächst das Programm, wächst die Menge der Variablen und Datenstrukturen
  - zur Speicherung von Daten
  - zur Kontrolle des Programmflusses
  - zum Abspeichern von Zuständen
  - zum Verarbeiten von Ein- und Ausgaben
- Die zugrundeliegenden Elemente basieren oft auf denselben Strukturen
- Z.B. Punkt-Koordinaten: als Tupel oder als Liste? — ohne Festlegung sind beide möglich
- Objekte legen die **Syntax** einer Datenstruktur verbindlich fest

Warum Objekte — Das semantische Problem

Objekte werden auch verwendet, um die Semantik von Werten einer Datenstruktur eindeutig zu definieren. Haben wir uns darauf geeinigt, einen Punkt syntaktisch durch ein Tupel zu repräsentieren, so ist die **Bedeutung** der Werte dennoch nicht bekannt.

Ohne Objekt:

```
punkt_1 = (54.083336, 12.108811)
```

- Ist der erste Wert Latitude oder Longitude?
- Kartesisch oder radial?

Mit Objekt:

```
punkt_1.latitude  = 54.083336
punkt_1.longitude = 12.108811
```

- Attributnamen machen Bedeutung explizit

Warum Objekte — Das Verhaltensproblem

Objekte werden zudem verwendet, um die Funktionen zur Verarbeitung der Datenstrukturen direkt mit dieser zu bündeln, so dass die Datenstruktur nur jene Funktionen anbietet, welche auch sinnvoll anwendbar sind.

Ohne Objekte:

- Eine freie Funktion `distanz(a, b)` kann auf beliebige Daten angewendet werden
- Auf zwei Punkte genauso wie auf zwei Linien — kein Schutz
- Können die Daten mit der Funktion verarbeitet werden? Das prüft niemand
- Führt zu semantischen und logischen Fehlern

Mit Objekten:

- Die Operation `distanz()` gehört zur Klasse `Punkt`
- Die Klasse *kapselt* Daten und zulässige Operationen
- Falsche Anwendung wird verhindert

OBJEKTORIENTIERTE PROGRAMMIERUNG

Objektorientierte Programmierung (OOP) ist ein Programmierparadigma, das annimmt, dass ein Programm ausschließlich aus Objekten besteht, die miteinander kooperativ interagieren.

Jedes Objekt verfügt über Attribute (Eigenschaften) und Methoden (Verantwortlichkeiten).

- Die **Attribute** definieren Werte über den Zustand eines Objektes
- Die **Methoden** definieren die möglichen Zustandsänderungen (Handlungen) eines Objektes

Grundlagen der Objektorientierung

- Anstatt unübersichtlich viele verstreute Datenstrukturen und Funktionen zu benutzen, gruppieren wir diese in **Objekte**
- Die Struktur dieser Objekte wird in Form von **Klassen** definiert, die eine Art Bauplan darstellen
- Eine Klasse definiert:
  - welche **Attribute** (Eigenschaften) ein Objekt dieser Klasse besitzt
  - welche **Methoden** (Verantwortlichkeiten) ein Objekt dieser Klasse bereitstellt
- Beides wird zusammen in einer komplexen Datenstruktur gespeichert: Jedes Objekt stellt somit einen zusammengesetzten (komplexen) Datentyp dar

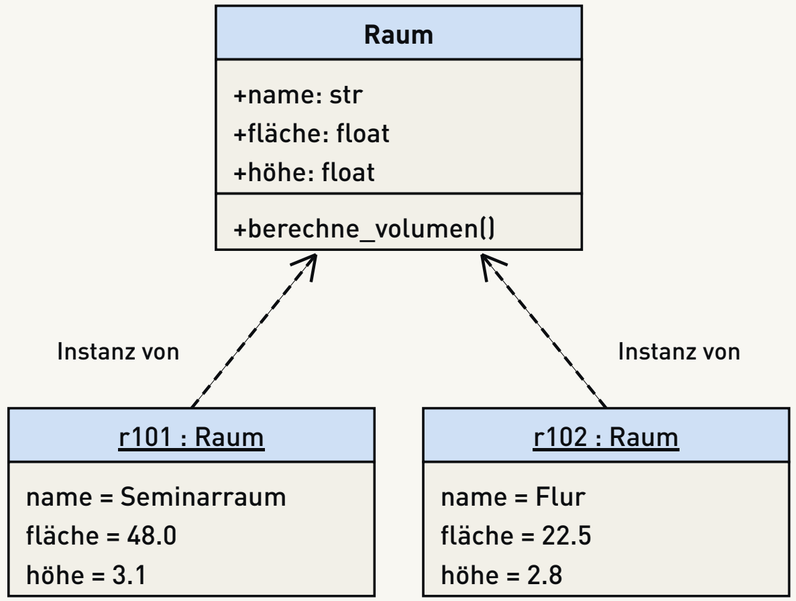
Klassen und Instanzen

Klasse = Bauplan (Entwurf)

- Definiert Struktur und Verhalten
- Keine Werte

Instanz = konkretes Objekt

- Entsteht nach dem Bauplan
- Hat eigene Attributwerte
- Viele Instanzen einer Klasse möglich



UML-KLASSENDIAGRAMME

Modellierung mit Objekten

- Unsere Wahrnehmung und unser Denken besteht darauf Objekte zu erkennen. Das setzt voraus, dass wir das Objekt von anderen Objekten trennen können (z.B. Stuhl und Tisch)
- Das Nachbilden dieser Wahrnehmung und dieses Weltverständnisses im Computer, nennt man *Modellierung*.
- Da unsere Wahrnehmung auf Objekten basiert, ist es naheliegend für die Modellierung auch Objekte zu nutzen.
- Die Objekte in der Welt interagieren, dieses Verhalten müssen wir auch modellieren.

> **MODELL**
>
> Ein Modell ist eine abstrakte, vereinfachte Abbildung eines Systems (meist aus der Realität).

Hörsaalfrage

Was haben diese Elemente gemein?



Hörsaalfrage

- Das sind alles *geometrische Objekte*
- Sie bestehen alle aus *Punkten*, die mit *Linien* verbunden sind
- Das können wir bei der Modellierung ausnutzen und Klassen so definieren, dass sie die *Gemeinsamkeit* ausnutzen



Objektorientierten Softwareentwurf

Im objektorientierten Softwareentwurf wird ein Programm so entworfen, dass es nur aus *Objekten* besteht.

- Der Softwareentwurf gleicht dabei stark dem Vorgehen von Ingenieuren beim Entwurf von Plänen
- Die übliche Modellierungssprache sind *UML Diagramme*.
- Man modelliert im Entwurf:
  - wie Objekte in Form von *Klassen* definiert sind,
  - welche *Attribute und Methoden* sie besitzen,
  - wie diese Klassen aufeinander aufbauen (*Vererbung*),
  - wie sie miteinander in statischer Beziehung stehen (*Assoziation, Aggregation, Komposition*)
  - und wie sie dynamisch interagieren (*Verhalten*)

UML — Entwurfssprache für Software

- *UML (Unified Modelling Language)* ist ein ISO-Standard zur Beschreibung von Softwareentwürfen
- Er wird genutzt für den Entwurf, Ausschreibung, Implementation und Dokumentation von:
  - Funktionalitäten
  - Strukturen
  - Businessprozessen
  - Nutzerinteraktionen

Der Standard (UML 2.5) umfasst 14 Diagrammtypen, die typischsten:

- *Klassendiagramme* (Für die Klassen- und Datenstruktur)
- *Aktivitätsdiagramme* (Für allgemeine Abläufe und Algorithmen)
- *Sequenzdiagramme* (Für Interaktionen)
- *Use-Case Diagramme* (Für Nutzer und Nutzfälle)

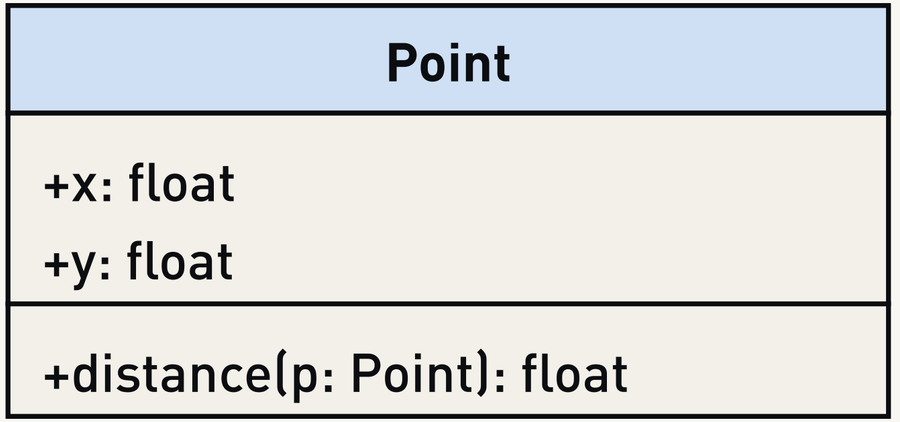
UML Klasse — Aufbau

Eine Klasse besteht aus drei Ebenen:

01 *Klassenname* (oben)

02 *Attribute* mit Datentyp (mitte)

03 *Methoden* mit Signatur (unten)



Attribut- und Methoden-Notation

Attribute:

```
attributname: Typ
```

Beispiele:

```
name: str
area: float
rooms: list[Room]
```

Methoden:

```
method(param: Typ): Rückgabetyp
```

Beispiele:

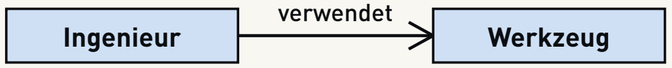
```
calculate_area(): float
get_rooms(): list[Room]
add_room(r: Room)
```

Objektbeziehungen

In Programmen stehen Objekte im Zusammenhang. Diese Beziehungen müssen im Entwurf **explizit** festgehalten werden:

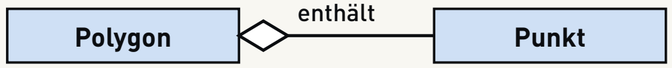
Assoziation

Ein Objekt *verwendet* ein anderes.



Aggregation

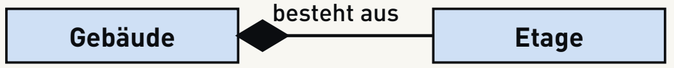
Ein Objekt *enthält* andere (lose).



Punkte können auch allein existieren

Komposition

Ein Objekt *besteht aus* anderen (strikt).



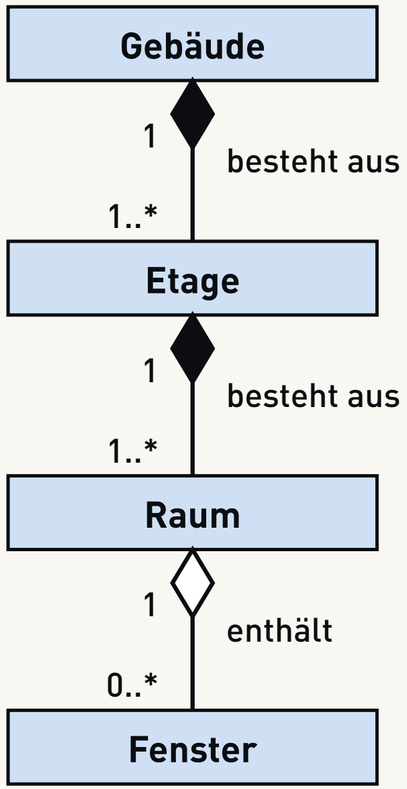
Ohne Gebäude ist eine Etage nicht sinnvoll

Multiplizität

Zahlen an den Linien geben die *Multiplizität* an — wie viele Objekte in Relation stehen:

Notation	Bedeutung
`1`	Genau eins
`0..*`	Null oder mehr
`1..*`	Ein oder mehr
`0..1`	Keins oder eins

Im Entwurf klären: Welche Richtung? Wie viele? Wer lebt von wem ab?



Kapselung, Vererbung, Polymorphismus

Kapselung

- Daten und zulässige Operationen gehören zusammen
- Nicht jedes Attribut soll von außen beliebig änderbar sein
- UML markiert Sichtbarkeit mit `+`, `#`, `–`

Vererbung

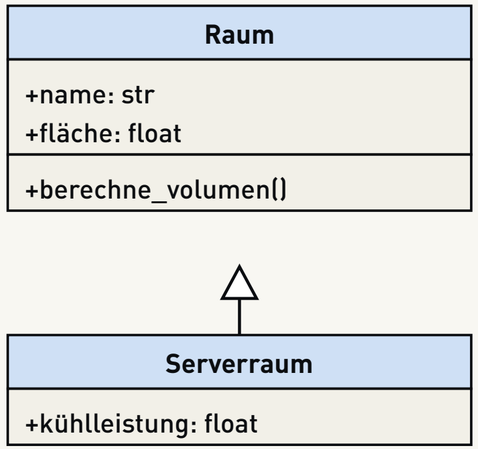
- Eine Unterklasse spezialisiert eine allgemeinere Oberklasse
- Beispiel: `Serverraum` ist ein spezieller `Raum`
- In UML: Generalisierung mit weißem Dreieck

Polymorphismus

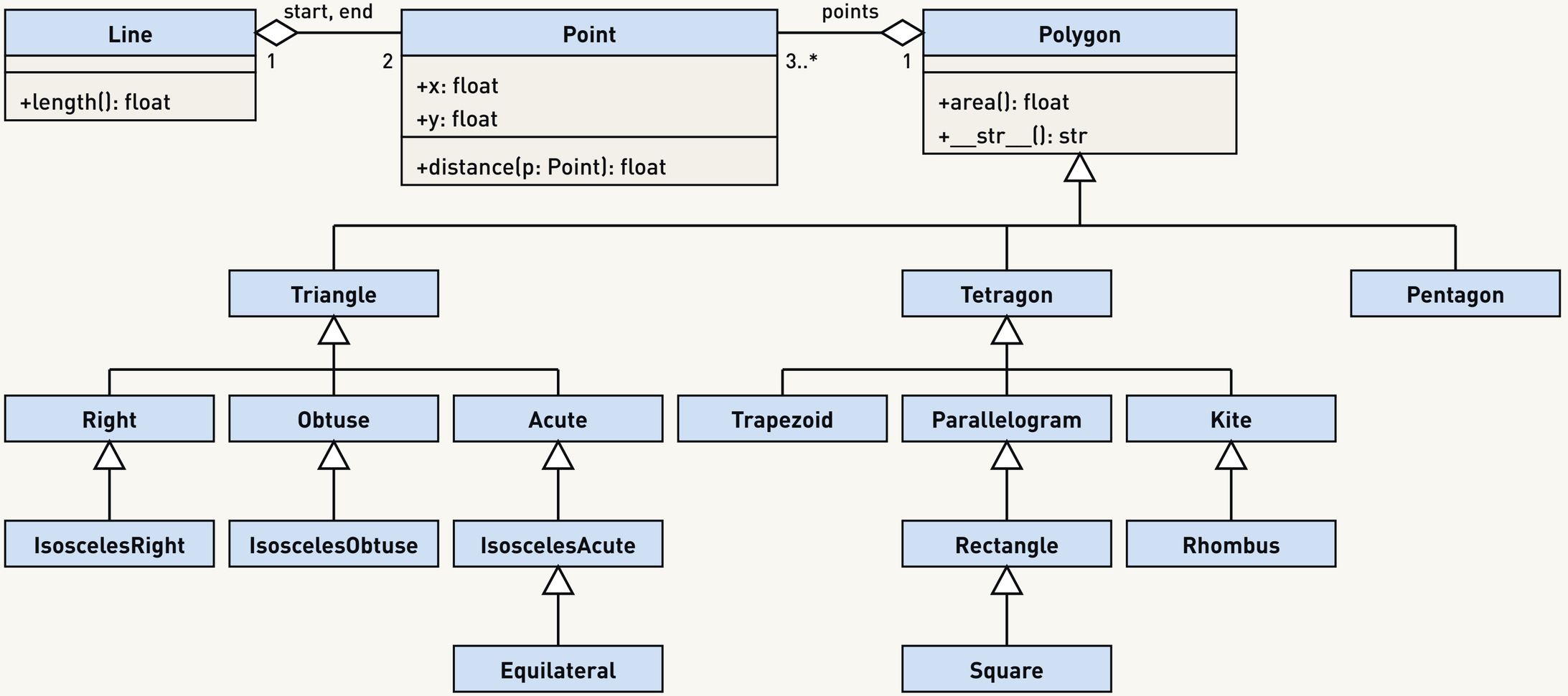
- Gleiche Methode, unterschiedliches Verhalten
- Beispiel: `fläche()` bei `Rechteck` und `Kreis`
- Wichtig für austauschbare Objekte mit gleichem Interface

Vererbung in UML

- Die Oberklasse `Raum` trägt, was alle Räume gemeinsam haben
- `Serverraum` ergänzt nur, was ihn besonders macht
- Die Linie mit weißem Dreieck zeigt von der Unterklasse zur Oberklasse



Vererbung — Geometrie-Hierarchie



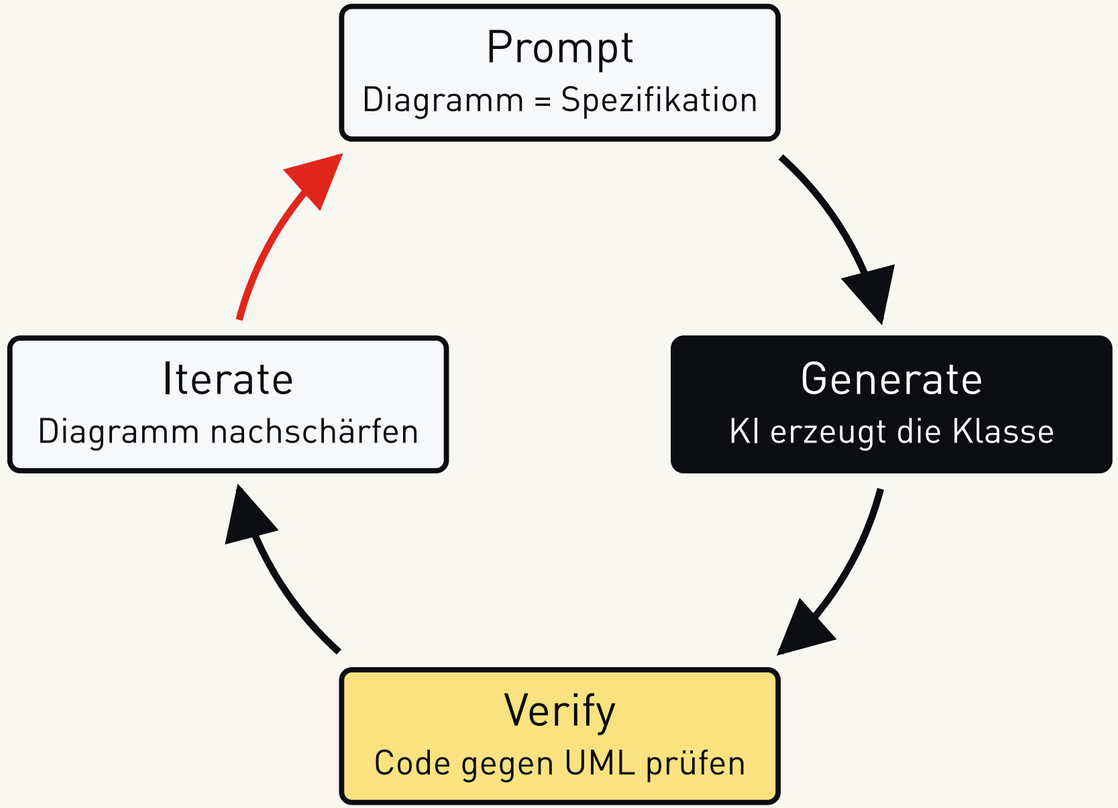
`area()` wird einmal in `Polygon` entworfen; alle 16 Unterklassen erben sie. Klassendiagramme werden schnell groß, zeigen aber das ganze Datenmodell auf einen Blick.

AGENTISCHER OBJEKTENTWURF

Design before you prompt

Das UML-Diagramm ist die Spezifikation. Der Code ist das Ergebnis.

- **Prompt:** Das Diagramm geht als Spezifikation an die KI
- **Generate:** Die KI erzeugt die Python-Klasse
- **Verify:** Der Code wird gegen das Diagramm geprüft
- **Iterate:** Abweichungen schärfen Diagramm und Prompt nach



UML-Diagramm als KI-Prompt

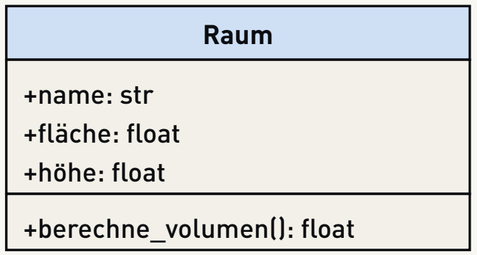
Ein vollständiges UML-Klassendiagramm enthält:

Klassennamen	**Attribute**	**Verantwortlichkeiten**	**Beziehungen**
eindeutig, aussagekräftig	Name + Typ z.B. `fläche: float`	Methoden mit Parametern und Rückgabetyp	Art und Multiplizität

> **Merksatz**
>
> Was nicht im Diagramm steht, wird die KI erfinden — oft falsch.

Mini-Beispiel: Diagramm als Prompt

UML-Fragment



Prompt-Fragment

```
Implementiere die Klasse Raum.
Attribute: name: str,
  fläche: float, höhe: float
Methode: berechne_volumen() -> float
Erfinde keine weiteren Attribute.
```

Verifizieren gegen das Diagramm

Agentic Workflow — Objekte entwerfen:

01 Lies die generierten Klassen gegen das UML-Diagramm

02 Prüfe: Sind alle Attribute vorhanden und korrekt benannt?

03 Prüfe: Sind alle Beziehungen korrekt implementiert?

04 Wenn nicht: Ergänze das Diagramm und prompte erneut

Mini-Beispiel: Verifikation

Generierter Code

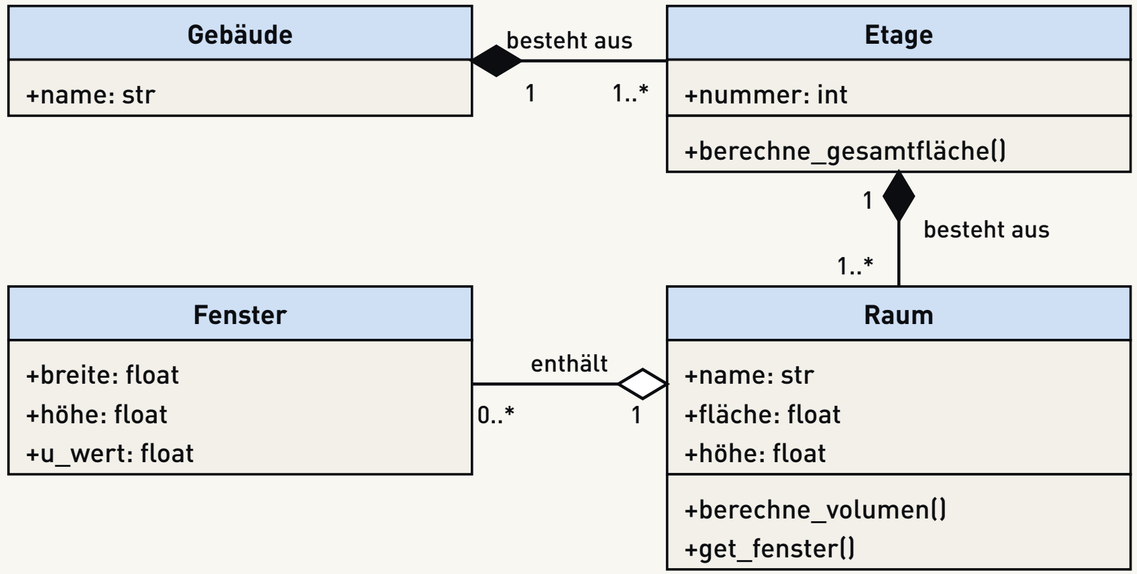
```python
class Raum:
    def __init__(self, name, fläche, höhe):
        self.name = name
```

Prüfung gegen das Diagramm

Diagramm	Code
`name: str`	vorhanden, Typ fehlt
`fläche: float`	wird nicht gespeichert
`höhe: float`	wird nicht gespeichert
`berechne_volumen()`	fehlt

Ergebnis: unvollständig — Prompt oder Diagramm nachschärfen

Domänenzerlegung: Gebäude



- Entitäten: `Gebäude`, `Etage`, `Raum`, `Fenster`
- Komposition: Gebäude → Etage → Raum
- Aggregation: Raum → Fenster
- Attribute und Methoden stehen im Diagramm, nicht erst im Code

BIM-Beispiel — Schritt 1: Anforderung formulieren

Auftraggeber formuliert:

> „Ein Gebäude hat mehrere Räume. Jeder Raum hat einen Namen, eine Fläche und eine Liste von Fenstern. Ein Fenster hat eine Breite und Höhe. Das Programm soll die Gesamtfensterfläche eines Raumes berechnen."

Ingenieursfragen für den Entwurf:

- Welche *Klassen* brauchen wir?
- Welche *Attribute* mit welchen Typen?
- Welche *Methoden* mit Signaturen?
- Welche *Beziehungen* zwischen Klassen?

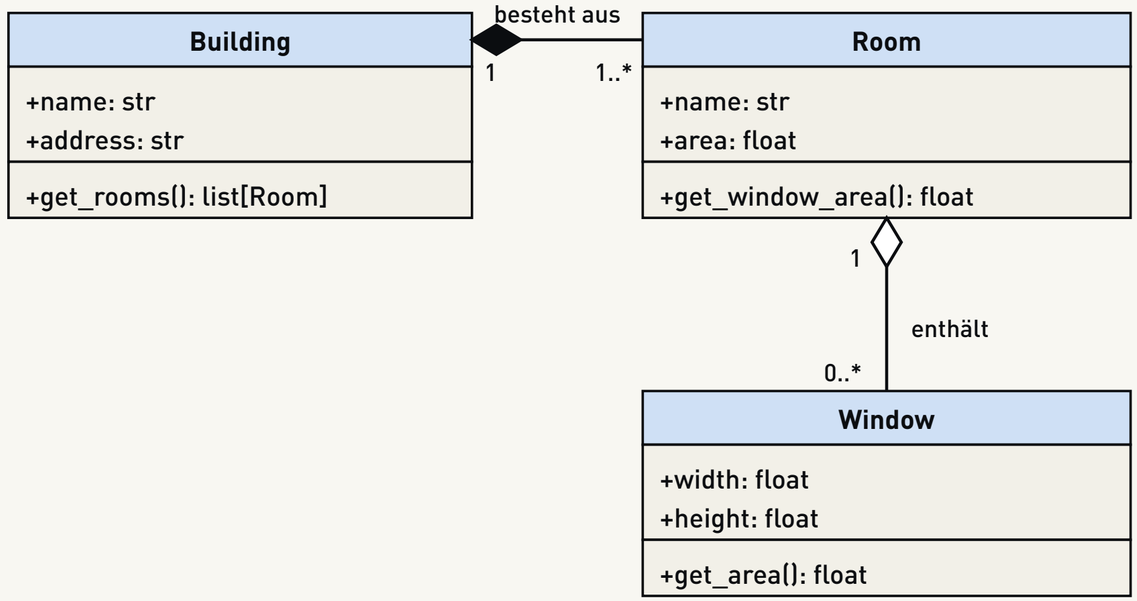
→ Diese Fragen beantwortet das UML-Diagramm

Hörsaalfrage

Welche Entitäten hat ein Gebäude?


© Midjourney: Entities of a Building

BIM-Beispiel — Schritt 2: Anforderung → UML



- „Gebäude hat Räume“ → Komposition `1` zu `1..*`
- „Raum hat Liste von Fenstern“ → Aggregation `1` zu `0..*`
- „Gesamtfensterfläche berechnen“ → Methode in `Room`

BIM-Beispiel — Schritt 3: UML → KI-Prompt → Verifizieren

KI-Prompt (aus UML abgeleitet):

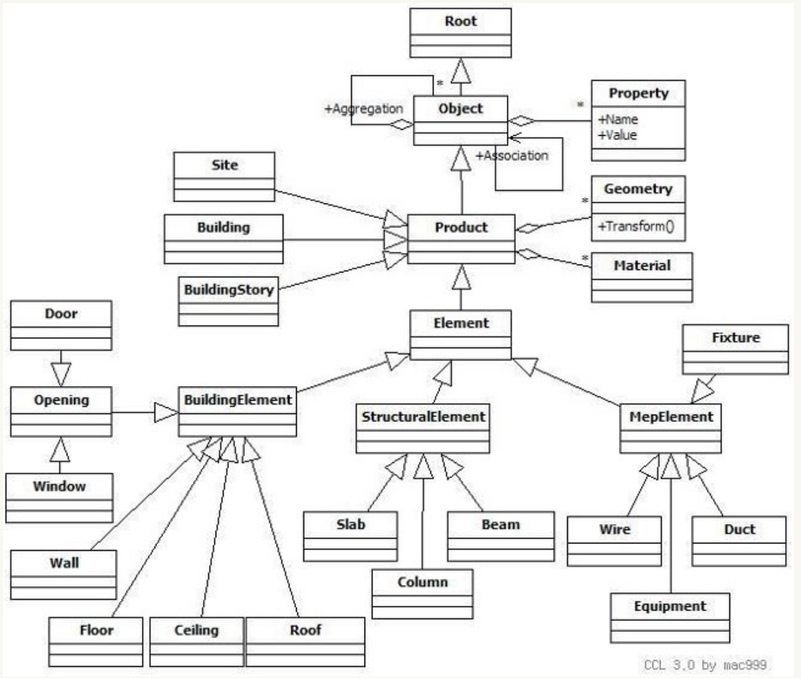
> Erstelle Python-Klassen für `Building`, `Room` und `Window`. `Building` hat `name: str`, `address: str`, eine Liste von `Room`-Objekten und `get_rooms() -> list[Room]`. `Room` hat `name: str`, `area: float`, eine Liste von Fenstern und `get_window_area() -> float`. `Window` hat `width: float`, `height: float` und `get_area() -> float`.

Verifikation gegen UML:

- ✓ Alle drei Klassen vorhanden?
- ✓ Attribute mit korrekten Typen?
- ✓ Methoden-Signaturen korrekt?
- ✓ Beziehungen implementiert?
- ✓ Multiplizität berücksichtigt?

Anwendungsbeispiel — BIM (Building Information Models)

- Bauzeichnungen werden heutzutage als *IFC (Industry Foundation Classes)* gespeichert
- *IFC* ist ein objektorientiertes Modell mit Vererbung, Spezialisierung und Generalisierung
- Es wird u.a. in *UML* dokumentiert



Vereinfachtes IFC-Klassenmodell (mac999, CCL 3.0)

Querverbindungen

S03 — `03c_Softwareentwurf`

Vorgehensmodelle: Entwurf vor Implementierung

S06 — `06b_Objects` (Implementierung)

Python-Syntax für dieselben Konzepte: `class`, `__init__`, `self`, Attribute, Methoden

S08 — `08a_UnitTest`

Tests verifizieren, ob die Python-Implementierung dem UML-Entwurf entspricht

QUESTIONS